

Tracker — Project Overview

Prepared for: Senior Stakeholder Review **Date:** September 2026 **Status:** Phase 7 of 8 Complete — Production-Ready Core, Analytics Layer Pending

1. Executive Summary

Tracker is a multi-tenant affiliate tracking SaaS platform — a direct competitor in the Trackier / Everflow / Tune segment of the market. It captures clicks, attributes conversions, manages an append-only money ledger, and reports performance across publishers, advertisers, and partners.

The platform is a **two-repo system**:

- `api-backend/` — Node.js + TypeScript server, split across five segregated HTTP surfaces
- `frontend/` — React 18 single-page application

Headline numbers:

- ~**163** frontend page modules across 4 role groups
- 50+** backend route modules on the dashboard surface alone
- 62** database migrations
- 6** BullMQ worker queues for async processing
- 8** external integrations (Supabase Auth, MaxMind, Sentry, Claude, Facebook CAPI, etc.)

The codebase is **production-shaped**: real auth, real multi-tenancy, real Redis-backed hot path, real money ledger. What remains is the analytics performance layer (Phase 8 — ClickHouse ingestion).

2. What the Project Does

Tracker is built to do five things, end-to-end:

Capability	Description
Click Tracking	A user clicks an affiliate link → Tracker records the click with full attribution context (publisher, offer, geo, device, sub-IDs) and 302-redirects to the advertiser.
Conversion Attribution	When a conversion happens (server postback, pixel, iframe, or API), Tracker matches it back to the original click and credits the publisher.
Money Ledger	Every payout, reversal, bonus, charge, or adjustment is recorded as an append-only ledger entry. Balances are computed, never stored.
Reporting	Dashboards and exports show clicks, conversions, revenue, EPC, CR, and ROI sliced by any combination of dimensions (offer, publisher, country, device, etc.).
Multi-Tenant Operations	Each "network" (tenant) is fully isolated — every query filters by <code>network_id</code> ; each tenant can manage users, offers, publishers, advertisers, partners, smart links, domains, API keys, and reports.

In short: **run an affiliate network as SaaS.**

3. Main Modules and Their Purpose

3.1 Backend Surfaces

The backend runs as **five independent HTTP servers**, each on its own port, each with a distinct concern:

Port	Surface	Stack	Purpose
4001	Dashboard	Express	Internal UI for network admins/managers — CRUD, reports, settings
4002	Tracking	Fastify	Public hot path — <code>/click</code> , <code>/postback</code> , <code>/pixel</code> , <code>/iframe</code> , <code>/sl</code>
4003	Public API	Express	REST API for external integrations — API-key authenticated
4004	Platform Admin	Express	Internal operations — manage tenants, platform-wide settings
4005	Workers	(no HTTP)	BullMQ queue consumers — async processing

This separation matters: the **tracking surface** has a latency budget measured in single-digit milliseconds; isolating it lets it scale independently of the dashboard's heavy CRUD workload.

3.2 Frontend Role Groups

The SPA is divided into **four role-scoped route groups**, all lazy-loaded:

1. **Admin** — Platform-level operations (network management, platform admin tasks)
2. **Network** — Network admins / managers (offers, users, advertisers, reports, settings)
3. **Publisher** — Publisher-facing views (their offers, links, stats, payouts)
4. **Advertiser** — Advertiser-facing views (their campaigns, conversions, spend)

Routing is gated by `ProtectedRoute` with role-based redirects (`ROLE_HOME` per role).

3.3 Backend Domain Modules

Within the dashboard surface, ~50 route modules cover the domain:

- **Offers** — CRUD, caps (global / daily / monthly), targeting, pixels, smart-link variants
- **Affiliates (Publishers / Partners / Advertisers)** — onboarding, status, caps, postbacks
- **Reports** — clicks, conversions, revenue, by-offer / by-publisher / by-country / by-device
- **Money / Ledger** — append-only ledger entries, balance computation, payout workflows
- **Tracking Domains** — per-tenant custom domains for affiliate links
- **Smart Links** — rotator links that pick the best-matching offer per click
- **API Keys** — generation, revocation, scope management
- **Users / Roles** — RBAC (admin, manager, custom roles)
- **Settings** — network-level configuration
- **Integrations** — Facebook CAPI, Sentry, MaxMind keys, webhook URLs

3.4 Worker Queues

Six BullMQ worker queues handle work that must not block HTTP:

Queue	Job
click-persist	Write click events to Postgres in batches
postback-process	Validate and attribute inbound postbacks
fraud-scan	Run fraud heuristics on incoming clicks

retention-purge	Enforce data-retention policies
capi-dispatch	Send Facebook Conversions API events
feed-sync	Pull/push external offer feed networks

4. How the Overall Workflow Works

4.1 Click Flow (the core)

```

End user clicks https://track.example.com/click?offer_id=X&subid1=Y
|
▼
[Fastify :4002 – Tracking surface]
├─ Validate sub-IDs
├─ Geo-lookup via MaxMind (cached in Redis)
├─ Resolve network_id + offer_id (Redis-cached, NOT Postgres)
├─ Generate click_id (uuid)
├─ Enqueue click event → Redis (lpush click-persist)
├─ 302-redirect to advertiser's offer URL (with click_id injected)
|
▼ (background, < 50ms p99)
[Worker :4005 – click-persist queue]
├─ Batched insert into `clicks` table
├─ Run fraud-scan
├─ Emit analytics event (Phase 8 → ClickHouse)

```

Key invariant: the synchronous hot path **never** touches Postgres. Click persistence is async; the user's redirect happens in milliseconds.

4.2 Conversion Flow

When the advertiser's system or pixel fires, Tracker receives a postback (server-to-server), pixel beacon, or iframe ping:

```

POST /postback?click_id=...&payout=10.00&status=approved
|
▼
[Tracking surface]
├─ Look up click by click_id (Redis cache → Postgres fallback)
├─ Validate offer + publisher + payout
├─ Append ledger entry (event-driven, NOT stored as a mutable row)
├─ Fire downstream:
|   ├── CAPI dispatch (Facebook)
|   ├── Webhook fan-out
|   └─ Update conversion aggregates
└─ Return 200 to advertiser

```

4.3 Auth Flow

Four distinct auth paths serve different surfaces:

Surface	Auth
Dashboard UI	Supabase JWT (HTTP-only refresh cookie)
Platform Admin UI	Own JWT, separate secret
Public API	API Key (tracker_pk_*, hashed at rest)
Tracking	None (public), but rate-limited + fraud-scanned

Login → Supabase issues access token → frontend stores access token in localStorage (`tracker.session.v2`), refresh in httpOnly cookie → API client auto-refreshes on 401.

4.4 Reporting Flow

Reports are computed server-side via SQL aggregations over the `clicks` and `conversions` tables. The frontend sends filter objects (date range, dimensions, group-by) which are validated by zod schemas on the backend. Results are returned paginated.

5. What Has Been Completed

Phase 0 — Foundations

- Monorepo split, ESM modules, raw pg, zod, envelope responses
- Five-surface separation
- Supabase Auth integration
- Database migrations (0–62)

Phase 1 — Click Tracking Core

- `/click` endpoint with Redis-cached resolution
- Click-persist worker, batched inserts
- Geo (MaxMind) integration
- 302 redirect with `click_id` injection

Phase 2 — Conversion Attribution

- `/postback`, `/pixel`, `/iframe` endpoints
- Click → conversion matching by `click_id`
- Append-only ledger (event-driven)
- Conversion aggregates

Phase 3 — Affiliates & Offers

- Offer CRUD with global/daily/monthly caps (Redis-enforced)
- Publisher / Advertiser / Partner management
- Targeting (geo, device, OS)
- Smart links (rotator logic)

Phase 4 — Money & Payouts

- Ledger entry model
- Balance computation
- Payout workflows
- Reversal / chargeback handling

Phase 5 — Reporting

- Server-side aggregation queries
- Filterable reports (date, offer, publisher, country, device)
- Export endpoints
- Frontend report pages for all three roles

Phase 6 — Multi-tenancy Polish

- Tenant resolution by host
- API key generation / revocation
- Per-network settings
- RBAC for dashboard users

Phase 7 — Integrations & Operations

- Facebook CAPI dispatch worker
- Fraud-scan heuristics
- Webhook fan-out
- Sentry error reporting
- Email (Nodemailer) for transactional notifications
- ngrok for local webhook testing
- Smart-link persistence (Postgres-backed)

6. Approximate Completion Percentage

Area	% Complete	Notes
Click tracking	100%	Production-shaped, includes caps and fraud
Conversion attribution	100%	All entry points (postback/pixel/iframe/API)
Money ledger	100%	Append-only, balance computed
Offers + caps	95%	Targeting rules complete; advanced A/B variants deferred
Affiliates (Pub/Adv/Partner)	100%	CRUD + status + caps
Smart links	90%	Rotator works; analytics on variant performance deferred
Reporting	85%	Core slices done; cohort analysis deferred
Multi-tenancy	100%	network_id on every query
Auth & RBAC	100%	Four paths, role-gated routes
Integrations	90%	CAPI, Sentry, MaxMind, email done; some feed networks pending
Analytics layer (Phase 8)	0%	ClickHouse ingestion not started
UI test coverage	0%	No frontend tests exist
Backend test coverage	~30%	Smoke tests on hot paths only

Overall platform completeness: ~88%

The missing 12% is concentrated in two areas: the high-volume analytics ingestion pipeline (a known Phase 8 deliverable) and automated test coverage.

7. What Is Still Pending

7.1 Phase 8 — Analytics Layer (the big remaining item)

The codebase already declares the seam: `AnalyticsWriter` is an interface, not a concrete store. Today, click/conversion events are written to Postgres. Phase 8 swaps in ClickHouse behind the same interface — no caller rewrites.

Concrete work:

- Stand up ClickHouse (or compatible columnar store)
- Implement `AnalyticsWriter` against ClickHouse
- Backfill historical click/conversion data
- Build time-series report queries on the new store
- Migrate heavy aggregations off Postgres

This is the highest-leverage remaining work because today's reporting scales linearly with click volume against Postgres; ClickHouse makes it log-linear.

7.2 Test Coverage

- **Backend:** Smoke tests exist on hot paths (`/click` , `/postback`); full integration coverage and unit tests on services are missing
- **Frontend:** Zero automated tests — all validation is manual

7.3 Operational Polish

- Some known issues documented in `docs/claude/KNOWN_ISSUES.md` (e.g., two brand-name sources, no frontend tests, shadow Supabase role, trust-proxy configuration)
- Documentation should be updated as Phase 8 lands and any of the known issues are resolved

7.4 Deferred Features

- Cohort retention analysis
 - Offer A/B variant testing
 - Advanced fraud ML models (today uses heuristics)
 - Self-serve signup (currently admin-provisioned)
-

8. Current UI and Functionality Status

8.1 Frontend Status

The SPA is **fully navigable** across all four role groups. Every page that exists in the route map renders, lazy-loads, and is wired to the API.

What's working end-to-end (UI → backend → data):

- Login, session persistence, auto-refresh, logout
- Dashboard shell with per-role navigation
- Offers list / detail / create / edit

- Publishers list / detail
- Advertisers list / detail
- Partners list / detail
- Tracking domains
- Smart links
- Reports (clicks, conversions, revenue) with filters
- Settings (network-level)
- API key management
- User management
- Theme toggle (light / dark)

What's present in routing but in varying states of polish:

- Some advanced admin views (platform-wide analytics, billing)
- Export UX (CSV downloads work, scheduled email exports not built)

8.2 Backend Status

Production-shaped for:

- All tracking endpoints (/click , /postback , /pixel , /iframe , /sl)
- Auth + refresh for dashboard and platform admin
- API key issuance + revocation + scope check
- Offer CRUD + cap enforcement
- Ledger append + balance computation
- Reporting aggregations
- Webhook fan-out
- BullMQ worker processing (6 queues operational)

Operational:

- Multi-tenancy enforced (network_id filter on every query)
- Rate limiting on public endpoints
- Structured error envelopes
- Sentry integration for error capture
- Logging via pino
- Graceful shutdown handling on all five surfaces

Not yet wired:

- ClickHouse ingestion (Phase 8)
- Automated retry/recovery for failed worker jobs beyond BullMQ defaults
- Horizontal-scale considerations (today's Redis is single-node)

9. Risks and Watch Items

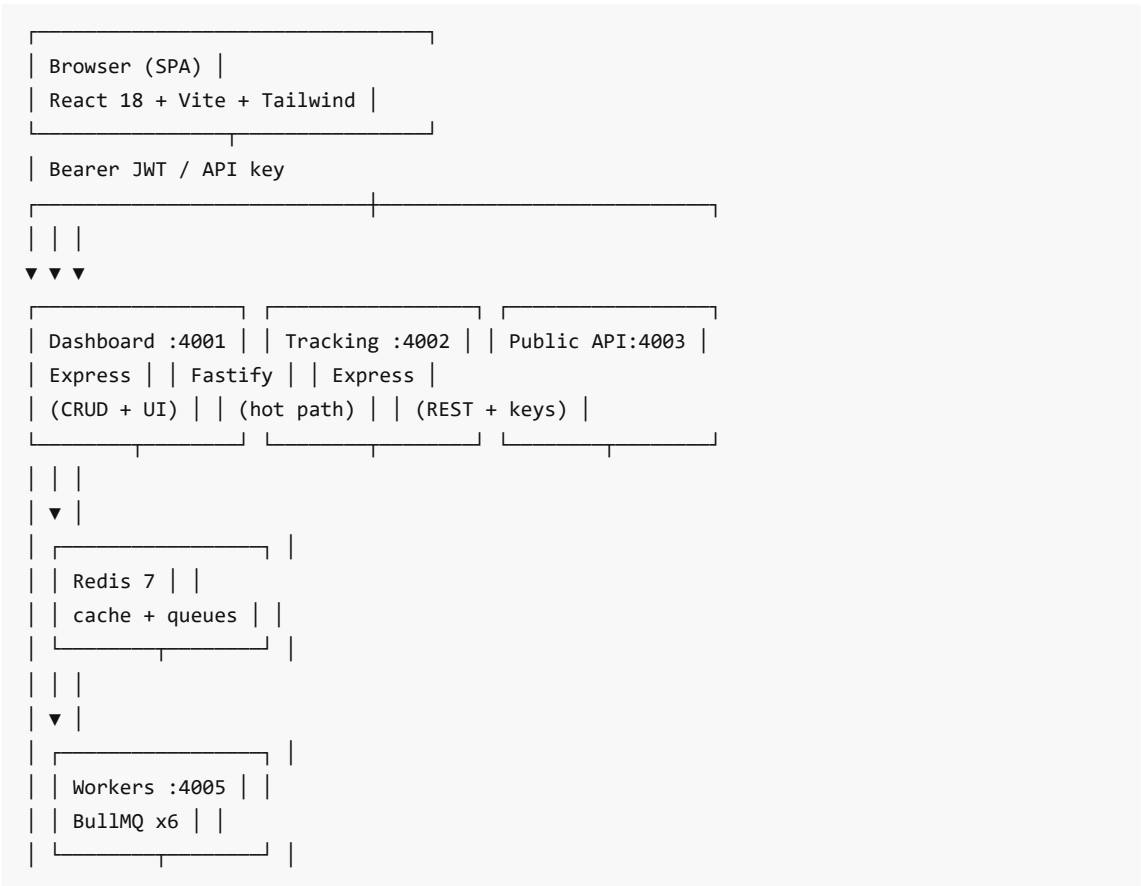
Risk	Severity	Mitigation
Postgres analytics scale	High	Phase 8 ClickHouse migration already designed for
No automated tests	High	CI currently runs typecheck + build only; manual QA
Shadow Supabase role enum	Medium	Known issue; one-time fix when role model stabilizes

Money as text in DB	Low (intentional)	Preserves decimal precision; UI formats on render
Brand-name duplication (two files)	Low	Single config-file edit on both sides when renaming
Redis single-node	Medium	Acceptable for current scale; multi-node for production scale

10. Recommended Next Steps

1. **Phase 8 — ClickHouse analytics layer** (highest leverage)
 - Stand up ClickHouse cluster
 - Implement `AnalyticsWriter` against it
 - Migrate reporting aggregations
 - Backfill historical data
2. **Test coverage** — at minimum, smoke-test every route module and worker queue
3. **Resolve the small pile of known issues** documented in `docs/claude/KNOWN_ISSUES.md` (mostly cosmetic / config)
4. **Operational hardening** — multi-node Redis, retry policies, observability dashboards
5. **Deferred features** — cohort analysis, A/B variants, self-serve signup

11. Architecture at a Glance



| | |
▼ ▼ ▼

Postgres 16 (62 migrations)
network_id on every row

12. Conclusion

Tracker is a **substantially complete affiliate tracking platform**. The hard parts — multi-tenant click tracking with low latency, append-only money ledger, conversion attribution, reporting, and a four-role UI — are all in place and operate against real infrastructure.

The remaining work is concentrated and well-scoped: the Phase 8 ClickHouse analytics layer is the largest item, and test coverage is the second. With those two delivered, the platform is feature-complete for the Trackier / Everflow competitive set and ready for production scale-out.

Bottom line for senior review: the platform is production-shaped and largely complete; the known gaps are well-understood and pre-planned in the phase roadmap.